Rapport · Lab 11 : Rendu du dossier de conception

Étape 11.1 · Exporter le schéma complet (tables, vues, fonctions, triggers, droits)

Question 1 : vérifier que la base queenbee contient tous les labs (6 à 10)

> *Vérifier que la base queenbee contient les objets des labs 6 à 10 avant d'exporter le schéma. Les fichiers SQL doivent passer avec -v ON_ERROR_STOP=1 (le script s'arrête à la première erreur).*

Réponse. on a exécuté sur la base de travail queenbee les fichiers SQL des labs 6, 8, 9 et 10. Les chemins sont ceux de la racine du dossier (sans jour2/) : ces exécutions datent donc d'avant le rangement de l'étape 11.3. Aucune erreur n'apparaît : seulement des CREATE, ALTER, GRANT, INSERT et des notices « n'existe pas, poursuite du traitement », normales avec DROP … IF EXISTS sur des objets pas encore créés. Les labs 7 (volume.sql, denormalisation.sql) sont documentés dans le Rapport Lab 7.

- Lab 6, formation.sql : INSERT 0 8 (table à plat), INSERT 0 5 (cinq formateurs), INSERT 0 8 (huit suivis), puis CREATE FUNCTION et CREATE TRIGGER : les résultats attendus (capture 1).
- Lab 8, references.sql : six ALTER TABLE, puis 16 clés étrangères dont 2 en CASCADE (les deux clés de adhesion) et 4 en RESTRICT (ruche_id de mesure, inspection, recolte et alerte). Les 10 autres restent en NO ACTION : c'est le résultat attendu (capture 2).
- Lab 8, check.sql : quatre ALTER TABLE acceptés, donc aucune ligne existante ne viole les contraintes (capture 3).
- Lab 8, audit.sql : trois fonctions et quatre triggers, recolte_audit, utilisateur_audit, journal_ajout_seul et journal_sans_truncate (capture 4).
- Lab 8, roles.sql : création des rôles (DO), REVOKE puis GRANT, six vues et la fonction de validation (capture 5). INSERT 0 0 signifie qu'aucune ligne n'est ajoutée : pas de doublon.
- Lab 9, jeton.sql : deux tables et INSERT 0 30 (les trente jetons) (capture 6).
- Lab 10, corrections.sql : vue de carte réécrite, ALTER VIEW, REVOKE, politique de sécurité sur recolte (capture 7).

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f formation.sql
psql:formation.sql:2: NOTICE:  la table « suivi_formation » n'existe pas, poursuite du traitement
psql:formation.sql:2: NOTICE:  la table « formateur » n'existe pas, poursuite du traitement
psql:formation.sql:2: NOTICE:  la table « formation_plat » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
INSERT 0 8
CREATE TABLE
CREATE TABLE
INSERT 0 5
INSERT 0 8
CREATE FUNCTION
CREATE TRIGGER

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$
```

Capture 1 : exécution de formation.sql sur la base de travail queenbee

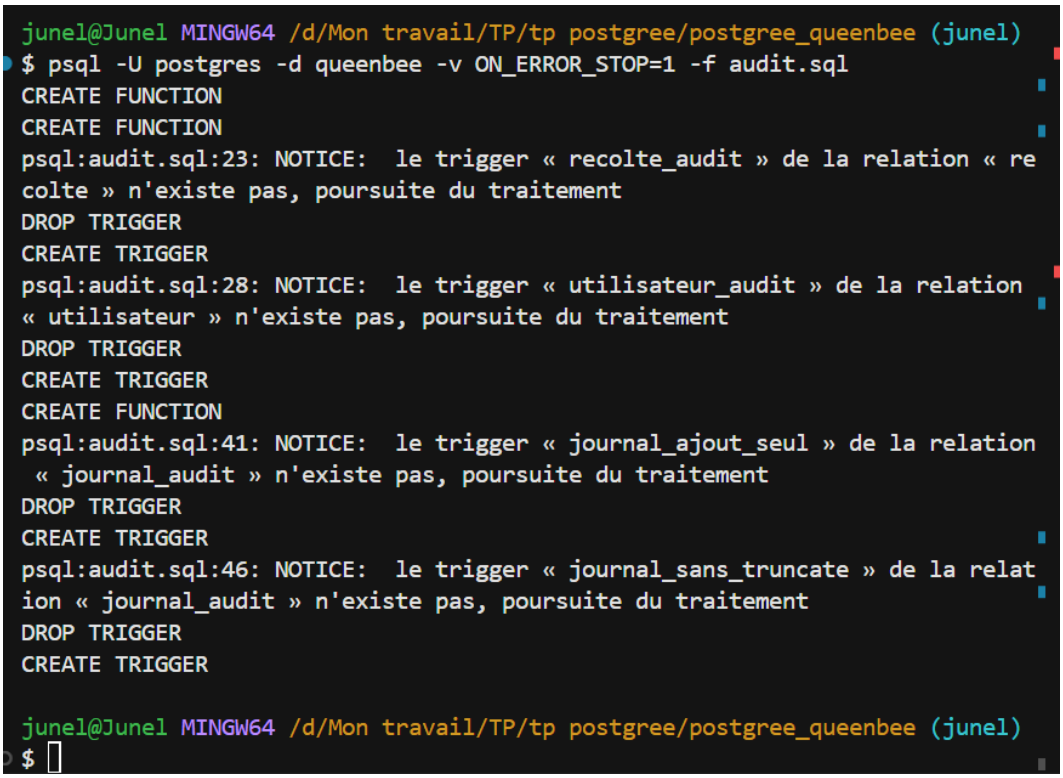
```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f references.sql
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
     table_      |              conname               | on_delete
-----------------+------------------------------------+-----------
 adhesion        | adhesion_cooperative_fk            | CASCADE
 adhesion        | adhesion_apicultrice_fk            | CASCADE
 cooperative     | cooperative_site_fk                | NO ACTION
 apicultrice     | apicultrice_site_fk                | NO ACTION
 ruche           | ruche_apicultrice_fk               | NO ACTION
 ruche           | ruche_site_fk                      | NO ACTION
 utilisateur     | utilisateur_site_fk                | NO ACTION
 utilisateur     | utilisateur_apicultrice_fk         | NO ACTION
 inspection      | inspection_apicultrice_fk          | NO ACTION
 recolte         | recolte_validee_par_fk             | NO ACTION
 suivi_formation | suivi_formation_apicultrice_id_fkey | NO ACTION
 suivi_formation | suivi_formation_formateur_id_fkey  | NO ACTION
 mesure          | mesure_ruche_fk                    | RESTRICT
 inspection      | inspection_ruche_fk                | RESTRICT
 recolte         | recolte_ruche_fk                   | RESTRICT
 alerte          | alerte_ruche_fk                    | RESTRICT
(16 rows)
```

Capture 2 : exécution de references.sql (2 CASCADE, 4 RESTRICT)

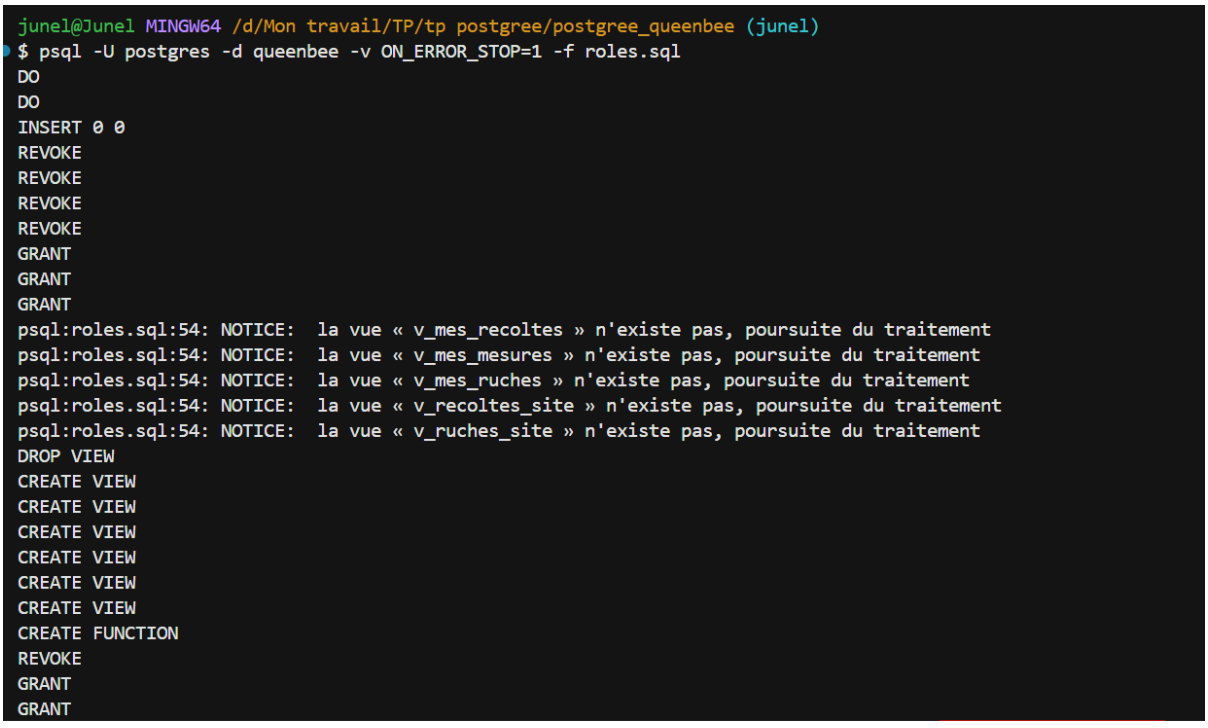
```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee -v ON_ERROR_STOP=1 -f check.sql
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$
```

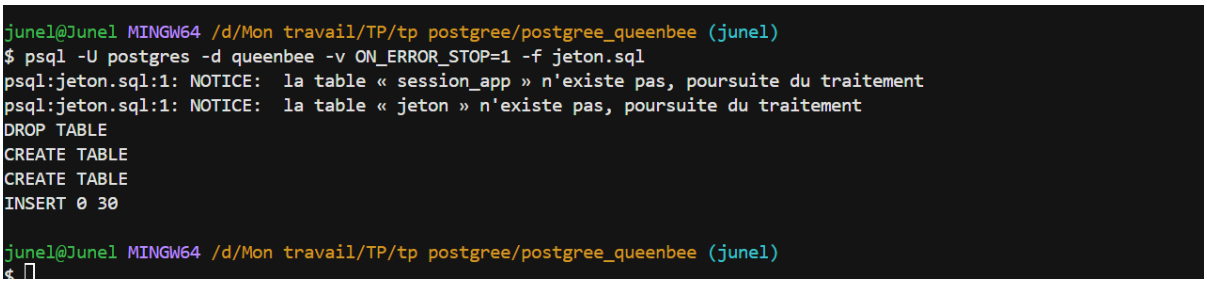
Capture 3 : exécution de check.sql



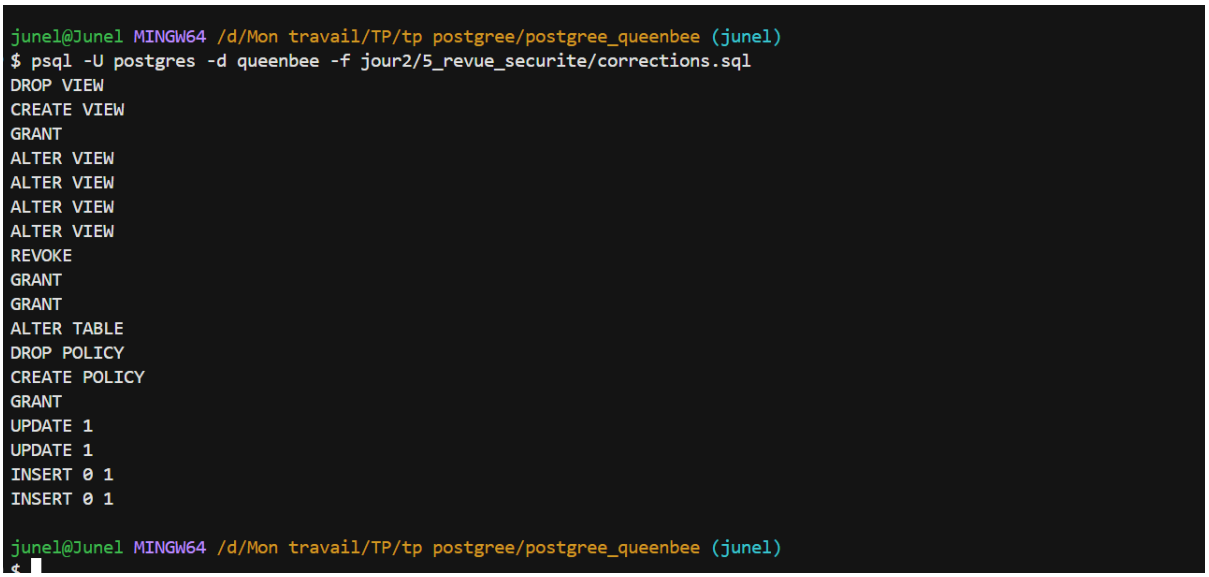
Capture 4 : exécution de audit.sql (3 fonctions, 4 triggers)



Capture 5 : exécution de roles.sql



Capture 6 : exécution de jeton.sql (INSERT 0 30)

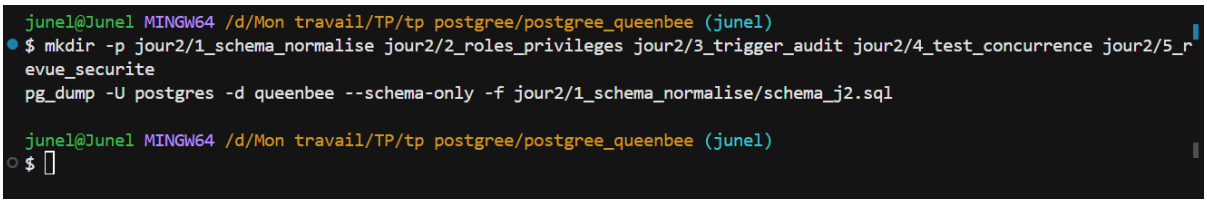


Capture 7 : exécution de corrections.sql sur la base de travail queenbee

Question 2 : créer les dossiers et exporter le schéma avec pg_dump

> *Créer les cinq dossiers de jour2/, puis exporter le schéma : pg_dump -U postgres -d queenbee --schema-only -f jour2/1_schema_normalise/schema_j2.sql.*

Réponse. une seule commande mkdir -p crée les cinq dossiers (1_schema_normalise, 2_roles_privileges, 3_trigger_audit, 4_test_concurrence, 5_revue_securite), puis pg_dump --schema-only écrit schema_j2.sql dans jour2/1_schema_normalise/. Aucun message d'erreur n'apparaît et l'invite revient : l'export a réussi. L'option --schema-only exporte la structure (tables, vues, fonctions, triggers, droits) mais pas les données : le fichier ne contient donc aucune donnée personnelle, ni la valeur de la clé de pseudonymisation. Les commandes ont été lancées depuis Git Bash (invite MINGW64).



Capture 8 : création des dossiers et export du schéma avec pg_dump

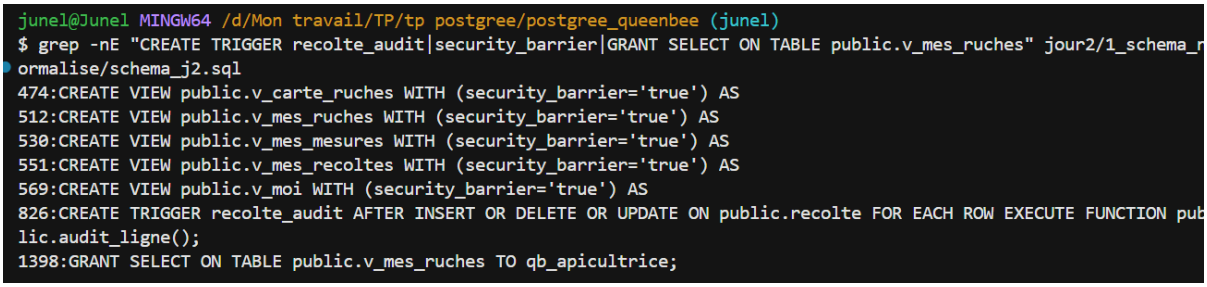
Question 3 : retrouver trois éléments dans schema_j2.sql

> *Ouvrir schema_j2.sql : on doit y trouver CREATE TRIGGER recolte_audit, security_barrier et GRANT SELECT ON TABLE public.v_mes_ruches.*

Réponse. on a cherché les trois motifs d'un coup avec grep -nE, plutôt que d'ouvrir le fichier à la main : c'est la même vérification, avec les numéros de ligne. Les trois sont présents.

- CREATE TRIGGER recolte_audit : ligne 826, AFTER INSERT OR DELETE OR UPDATE ON public.recolte FOR EACH ROW EXECUTE FUNCTION public.audit_ligne(). Le trigger d'audit est bien attaché à recolte.
- security_barrier : cinq vues sont déclarées WITH (security_barrier='true') : v_carte_ruches (ligne 474), v_mes_ruches (512), v_mes_mesures (530), v_mes_recoltes (551) et v_moi (569).
- GRANT SELECT ON TABLE public.v_mes_ruches : ligne 1398, accordé à qb_apicultrice. L'apicultrice lit sa vue et non la table ruche.

Le dump contient donc les triggers, les vues sécurisées et les droits : le schéma est complet.



Capture 9 : recherche des trois éléments dans schema_j2.sql

Étape 11.2 · Créer tests_audit.sql avec les tests de l'étape 8.3

Question 1 : lancer tests_audit.sql (points 2 à 4 de l'étape 8.3)

> *Créer tests_audit.sql avec les tests de l'étape 8.3 (points 2 à 4) et leurs résultats en commentaire.*

Réponse. on a lancé psql -U postgres -d queenbee -f tests_audit.sql, sans ON_ERROR_STOP car le fichier provoque des refus volontaires. Les trois tests donnent les résultats attendus.

- Test 2, modifier la quantité d'une récolte : UPDATE 1, puis le journal contient postgres | recolte | UPDATE | 12.50 | 14.00, la valeur avant puis après. Le TP annonce 12.5 | 14.0 : c'est le même résultat. La récolte modifiée est celle de SN-CASA-014 (nom de la ruche dans ma base).
- Test 3, effacer le journal : UPDATE, DELETE et TRUNCATE sont refusés par journal_interdit() avec « journal_audit est en ajout seul : UPDATE interdit », puis DELETE interdit et TRUNCATE interdit. Le nombre de lignes reste à 2 avant et après : rien n'a été effacé.
- Test 4, limite : deux ALTER TABLE passent. Le propriétaire (postgres) a désactivé le trigger puis l'a réactivé aussitôt. Un superutilisateur peut donc contourner la protection : c'est pourquoi le journal doit aussi être copié hors de la base, vers le SIEM du cahier des charges.

Les accents déformés dans la capture (« quantitÃ© ») viennent de l'encodage du terminal Git Bash, pas de la base.

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee -f tests_audit.sql
--- test 2 : modifier la quantitÃ© de la rÃ©colte de SN-CASA-014
UPDATE 1
  acteur  | table_cible | operation | avant | apres
----------+-------------+-----------+-------+-------
 postgres | recolte     | UPDATE    | 12.50 | 14.00
(1 row)


--- test 3 : effacer le journal
 count
-------
     2
(1 row)


psql:tests_audit.sql:15: ERREUR:  journal_audit est en ajout seul : UPDATE interdit
CONTEXT:  fonction PL/pgSQL journal_interdit(), ligne 3 à RAISE
psql:tests_audit.sql:16: ERREUR:  journal_audit est en ajout seul : DELETE interdit
CONTEXT:  fonction PL/pgSQL journal_interdit(), ligne 3 à RAISE
psql:tests_audit.sql:17: ERREUR:  journal_audit est en ajout seul : TRUNCATE interdit
CONTEXT:  fonction PL/pgSQL journal_interdit(), ligne 3 à RAISE
 count
-------
     2
(1 row)


--- test 4 : le propriÃ©taire peut dÃ©sactiver le trigger (limite)
ALTER TABLE
ALTER TABLE
```

Capture 10 : exécution de tests_audit.sql (tests 2, 3 et 4)

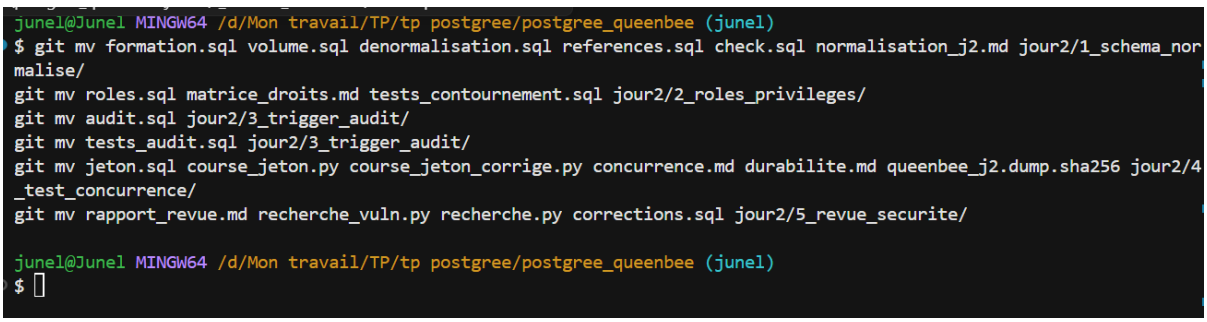
Étape 11.3 · Ranger les fichiers

Question 1 : ranger les fichiers dans jour2/ avec git mv

> *Ranger les fichiers dans les cinq dossiers. Si git mv répond not under version control, le fichier n'a jamais été ajouté : git add, puis recommencer.*

Réponse. les cinq commandes git mv ont réussi du premier coup : aucun message not under version control. Pour tests_audit.sql, on a utilisé git mv alors que le TP indique mv : sans conséquence, git suit le renommage. Voici le rangement obtenu.

Dossier	Fichiers rangés
1_schema_normalise	formation.sql, volume.sql, denormalisation.sql, references.sql, check.sql, normalisation_j2.md
2_roles_privileges	roles.sql, matrice_droits.md, tests_contournement.sql
3_trigger_audit	audit.sql, tests_audit.sql
4_test_concurrence	jeton.sql, course_jeton.py, course_jeton_corrige.py, concurrence.md, durabilite.md, queenbee_j2.dump.sha256
5_revue_securite	rapport_revue.md, recherche_vuln.py, recherche.py, corrections.sql



Capture 11 : rangement des fichiers avec git mv

Question 2 : contrôler le rangement avec git status

Réponse. git status --short liste 21 renommages (R) : les 21 fichiers déplacés, chacun dans le bon dossier (6, 3, 2, 6 et 4 fichiers par dossier). Trois remarques.

- check.sql, references.sql, roles.sql et tests_audit.sql sont marqués RM : renommés et modifiés depuis le dernier commit. Ces modifications seront prises par git add -A à l'étape 11.6.
- jour2/1_schema_normalise/schema_j2.sql est marqué ?? : fichier non suivi, ajouté lui aussi par git add -A.
- Aucun fichier .dump n'apparaît : la sauvegarde queenbee_j2.dump ne sera pas commitée, seule son empreinte queenbee_j2.dump.sha256 l'est, comme la consigne du lab 9 le demande.

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ git status --short
RM check.sql -> jour2/1_schema_normalise/check.sql
R  denormalisation.sql -> jour2/1_schema_normalise/denormalisation.sql
R  formation.sql -> jour2/1_schema_normalise/formation.sql
R  normalisation_j2.md -> jour2/1_schema_normalise/normalisation_j2.md
RM references.sql -> jour2/1_schema_normalise/references.sql
R  volume.sql -> jour2/1_schema_normalise/volume.sql
R  matrice_droits.md -> jour2/2_roles_privileges/matrice_droits.md
RM roles.sql -> jour2/2_roles_privileges/roles.sql
R  tests_contournement.sql -> jour2/2_roles_privileges/tests_contournement.sql
R  audit.sql -> jour2/3_trigger_audit/audit.sql
RM tests_audit.sql -> jour2/3_trigger_audit/tests_audit.sql
R  concurrence.md -> jour2/4_test_concurrence/concurrence.md
R  course_jeton.py -> jour2/4_test_concurrence/course_jeton.py
R  course_jeton_corrige.py -> jour2/4_test_concurrence/course_jeton_corrige.py
R  durabilite.md -> jour2/4_test_concurrence/durabilite.md
R  jeton.sql -> jour2/4_test_concurrence/jeton.sql
R  queenbee_j2.dump.sha256 -> jour2/4_test_concurrence/queenbee_j2.dump.sha256
R  corrections.sql -> jour2/5_revue_securite/corrections.sql
R  rapport_revue.md -> jour2/5_revue_securite/rapport_revue.md
R  recherche.py -> jour2/5_revue_securite/recherche.py
R  recherche_vuln.py -> jour2/5_revue_securite/recherche_vuln.py
?? jour2/1_schema_normalise/schema_j2.sql

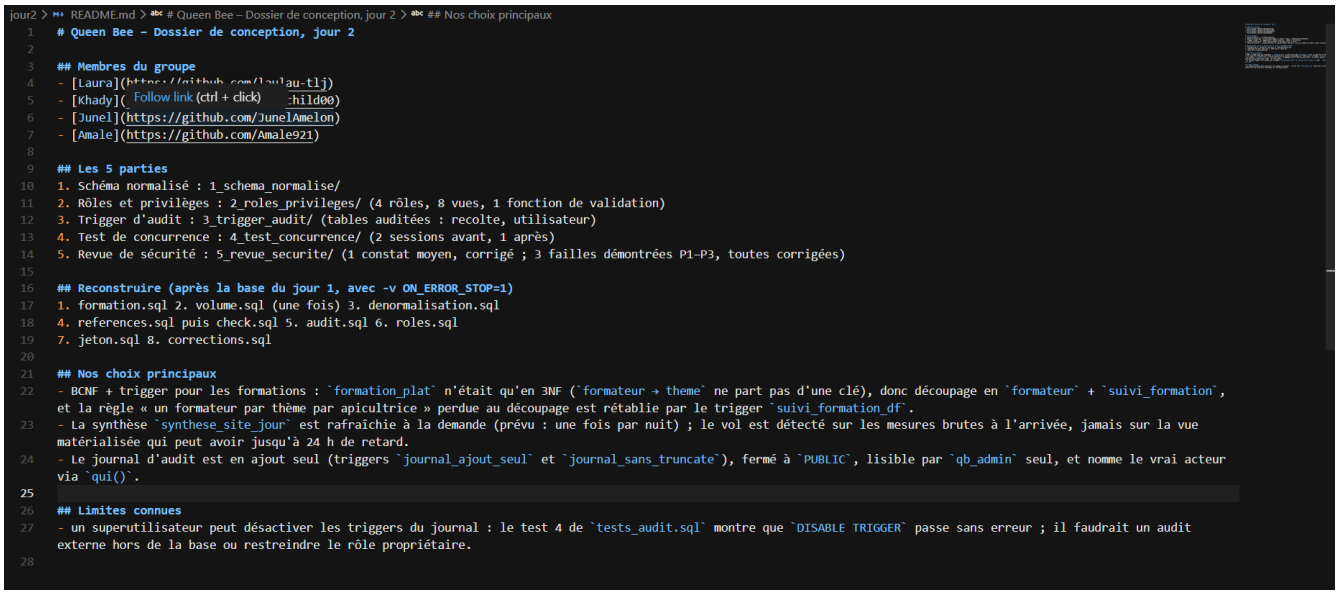
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$
```

Capture 12 : git status --short après le rangement

Étape 11.4 · Écrire jour2/README.md

Question 1 : écrire le README du dossier

> *Écrire jour2/README.md : les 5 parties, l'ordre de reconstruction, les choix principaux et les limites connues.*



```
jour2 > README.md > # Queen Bee – Dossier de conception, jour 2 > ## Nos choix principaux
# Queen Bee – Dossier de conception, jour 2

## Membres du groupe
- [Laura](https://github.com/laulau-tlj)
- [Khady]( Follow link (ctrl + click)      :hild00)
- [Junel](https://github.com/JunelAmelon)
- [Amale](https://github.com/Amale921)

## Les 5 parties
1. Schéma normalisé : 1_schema_normalise/
2. Rôles et privilèges : 2_roles_privileges/ (4 rôles, 8 vues, 1 fonction de validation)
3. Trigger d'audit : 3_trigger_audit/ (tables auditées : recolte, utilisateur)
4. Test de concurrence : 4_test_concurrence/ (2 sessions avant, 1 après)
5. Revue de sécurité : 5_revue_securite/ (1 constat moyen, corrigé ; 3 failles démontrées P1–P3, toutes corrigées)

## Reconstruire (après la base du jour 1, avec -v ON_ERROR_STOP=1)
1. formation.sql 2. volume.sql (une fois) 3. denormalisation.sql
4. references.sql puis check.sql 5. audit.sql 6. roles.sql
7. jeton.sql 8. corrections.sql

## Nos choix principaux
- BCNF + trigger pour les formations : `formation_plat` n'était qu'en 3NF (`formateur → theme` ne part pas d'une clé), donc découpage en `formateur` + `suivi_formation`,
et la règle « un formateur par thème par apicultrice » perdue au découpage est rétablie par le trigger `suivi_formation_df`.
- La synthèse `synthese_site_jour` est rafraîchie à la demande (prévu : une fois par nuit) ; le vol est détecté sur les mesures brutes à l'arrivée, jamais sur la vue
matérialisée qui peut avoir jusqu'à 24 h de retard.
- Le journal d'audit est en ajout seul (triggers `journal_ajout_seul` et `journal_sans_truncate`), fermé à `PUBLIC`, lisible par `qb_admin` seul, et nomme le vrai acteur
via `qui()`.

## Limites connues
- un superutilisateur peut désactiver les triggers du journal : le test 4 de `tests_audit.sql` montre que `DISABLE TRIGGER` passe sans erreur ; il faudrait un audit
externe hors de la base ou restreindre le rôle propriétaire.
```

Étape 11.5 · Tester la reconstruction dans une base vide

Question 1 : créer une base vide

> *Tester la reconstruction dans une base vide, en suivant le README : la base du jour 1, puis les huit fichiers du jour 2. Attendu : aucune ERROR.*

Réponse. on a créé une base vide queenbee_test avec dropdb --if-exists puis createdb. La notice « la base de données queenbee_test n'existe pas » est normale : c'est la première fois, et --if-exists évite l'erreur.

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ dropdb -U postgres --if-exists queenbee_test
createdb -U postgres queenbee_test
NOTICE:  la base de donnÃ©es Â« queenbee_test Â» n'existe pas, poursuite du traitement

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$
```

Capture 13 : création de la base vide queenbee_test

Question 2 : rejouer la base du jour 1 (schema.sql et donnees.sql)

Réponse. schema.sql crée les dix tables du jour 1 : dix notices « n'existe pas » du DROP TABLE IF EXISTS, puis une suite de CREATE TABLE. donnees.sql insère ensuite les données dans une transaction (BEGIN … COMMIT) : 4, 4, 8, 11, 8, 8, 2, 528, 6, 3 et 4 lignes. On y retrouve les 11 ruches du TP et 528 mesures, soit 11 ruches × 48 mesures. Dans donnees.sql, formation.sql vient ensuite : INSERT 0 8, INSERT 0 5, INSERT 0 8, la fonction et le trigger.

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f schema.sql
psql:schema.sql:3: NOTICE:  la table « alerte » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « recolte » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « inspection » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « mesure » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « adhesion » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « utilisateur » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « ruche » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « apicultrice » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « cooperative » n'existe pas, poursuite du traitement
psql:schema.sql:3: NOTICE:  la table « site » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

Capture 14 : rejeu de schema.sql (base du jour 1) sur queenbee_test

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f donnees.sql
BEGIN
INSERT 0 4
INSERT 0 4
INSERT 0 8
INSERT 0 11
INSERT 0 8
INSERT 0 8
INSERT 0 2
INSERT 0 528
INSERT 0 6
INSERT 0 3
INSERT 0 4
COMMIT

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/1_schema_normalise/formation.sql
psql:jour2/1_schema_normalise/formation.sql:2: NOTICE:  la table « suivi_formation » n'existe pas, poursuite du traitement
psql:jour2/1_schema_normalise/formation.sql:2: NOTICE:  la table « formateur » n'existe pas, poursuite du traitement
psql:jour2/1_schema_normalise/formation.sql:2: NOTICE:  la table « formation_plat » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
INSERT 0 8
CREATE TABLE
CREATE TABLE
INSERT 0 5
INSERT 0 8
CREATE FUNCTION
CREATE TRIGGER
```

Capture 15 : donnees.sql, puis formation.sql sur queenbee_test

Question 3 : rejouer les fichiers du jour 2, dans l'ordre du README

Réponse. tous les fichiers sont lancés depuis jour2/… avec -v ON_ERROR_STOP=1 : le script s'arrête à la première erreur, donc un déroulement complet prouve qu'il n'y en a pas. Les seules sorties parasites sont les notices « n'existe pas, poursuite du traitement », normales sur une base vide. Résultat pour chaque fichier :

Ordre	Fichier	Résultat observé	Conforme
1	formation.sql	INSERT 0 8, deux CREATE TABLE, INSERT 0 5 (cinq formateurs), INSERT 0 8 (huit suivis), fonction et trigger	Oui

Ordre	Fichier	Résultat observé	Conforme
2	volume.sql	INSERT 0 47520, puis ANALYZE	Oui
3	denormalisation.sql	vue matérialisée (SELECT 364) et son index, extension pgcrypto, table de clé (INSERT 0 1), vue d'export, table journal_audit	Oui
4	references.sql	six ALTER TABLE, 16 clés : 2 CASCADE et 4 RESTRICT	Oui
4	check.sql	quatre ALTER TABLE	Oui
5	audit.sql	trois fonctions et quatre triggers	Oui
6	roles.sql	rôles, INSERT 0 1, six vues, fonction valider_recolte, REVOKE et GRANT	Oui
7	jeton.sql	deux tables, INSERT 0 30	Oui
8	corrections.sql	vue de carte réécrite, quatre ALTER VIEW, REVOKE, politique recolte_apicultrice_isolement. Relancé une seconde fois : aucune notice, aucune erreur	Oui, rejouable

- denormalisation.sql donne SELECT 364 sur la base reconstruite, contre 365 sur la base de travail (Rapport Lab 7). L'écart d'une ligne s'explique vraisemblablement par la journée du 15 mars créée par la mesure de vol insérée à la main à l'étape 7.2 : elle ne fait partie d'aucun fichier de reconstruction.
- roles.sql donne INSERT 0 1 sur la base vide (le gestionnaire est ajouté à utilisateur) et INSERT 0 0 sur la base de travail (capture 5) : la ligne existait déjà et n'est pas dupliquée.

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/1_schema_normalise/volume.sql
INSERT 0 47520
ANALYZE

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/1_schema_normalise/denormalisation.sql
psql:jour2/1_schema_normalise/denormalisation.sql:4: NOTICE:  la vue matérialisée « synthese_site_jour » n'existe pas, poursuite du traitement
DROP MATERIALIZED VIEW
SELECT 364
CREATE INDEX
DROP MATERIALIZED VIEW
SELECT 364
CREATE INDEX
CREATE EXTENSION
psql:jour2/1_schema_normalise/denormalisation.sql:46: NOTICE:  la vue « v_export_bailleur » n'existe pas, poursuite du traitement
DROP VIEW
psql:jour2/1_schema_normalise/denormalisation.sql:47: NOTICE:  la table « cle_pseudonymisation » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
INSERT 0 1
REVOKE
CREATE VIEW
psql:jour2/1_schema_normalise/denormalisation.sql:75: NOTICE:  la table « journal_audit » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
REVOKE
```

Capture 16 : volume.sql, puis denormalisation.sql sur queenbee_test

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/1_schema_normalise/references.sql
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
     table_      |               conname               | on_delete
-----------------+-------------------------------------+-----------
 adhesion        | adhesion_cooperative_fk             | CASCADE
 adhesion        | adhesion_apicultrice_fk             | CASCADE
 cooperative     | cooperative_site_fk                 | NO ACTION
 apicultrice     | apicultrice_site_fk                 | NO ACTION
 ruche           | ruche_site_fk                       | NO ACTION
 ruche           | ruche_apicultrice_fk                | NO ACTION
 utilisateur     | utilisateur_site_fk                 | NO ACTION
 utilisateur     | utilisateur_apicultrice_fk          | NO ACTION
 inspection      | inspection_apicultrice_fk           | NO ACTION
 recolte         | recolte_validee_par_fk              | NO ACTION
 suivi_formation | suivi_formation_formateur_id_fkey   | NO ACTION
 suivi_formation | suivi_formation_apicultrice_id_fkey | NO ACTION
 mesure          | mesure_ruche_fk                     | RESTRICT
 inspection      | inspection_ruche_fk                 | RESTRICT
 recolte         | recolte_ruche_fk                    | RESTRICT
 alerte          | alerte_ruche_fk                     | RESTRICT
(16 rows)
```

Capture 17 : references.sql sur queenbee_test (16 clés : 2 CASCADE, 4 RESTRICT)

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/1_schema_normalise/check.sql
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/3_trigger_audit/audit.sql
CREATE FUNCTION
CREATE FUNCTION
psql:jour2/3_trigger_audit/audit.sql:23: NOTICE:  le trigger « recolte_audit » de la relation « recolte » n'existe pas, poursuite du traitement
DROP TRIGGER
CREATE TRIGGER
psql:jour2/3_trigger_audit/audit.sql:28: NOTICE:  le trigger « utilisateur_audit » de la relation « utilisateur » n'existe pas, poursuite du traitement
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:jour2/3_trigger_audit/audit.sql:41: NOTICE:  le trigger « journal_ajout_seul » de la relation « journal_audit » n'existe pas, poursuite du traitement
DROP TRIGGER
CREATE TRIGGER
psql:jour2/3_trigger_audit/audit.sql:46: NOTICE:  le trigger « journal_sans_truncate » de la relation « journal_audit » n'existe pas, poursuite du traitement
DROP TRIGGER
CREATE TRIGGER
```

Capture 18 : check.sql, puis audit.sql sur queenbee_test

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/2_roles_privileges/roles.sql
DO
DO
INSERT 0 1
REVOKE
REVOKE
REVOKE
REVOKE
GRANT
GRANT
GRANT
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_mes_recoltes » n'existe pas, poursuite du traitement
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_mes_mesures » n'existe pas, poursuite du traitement
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_mes_ruches » n'existe pas, poursuite du traitement
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_recoltes_site » n'existe pas, poursuite du traitement
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_ruches_site » n'existe pas, poursuite du traitement
psql:jour2/2_roles_privileges/roles.sql:54: NOTICE:  la vue « v_moi » n'existe pas, poursuite du traitement
DROP VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE VIEW
CREATE FUNCTION
REVOKE
GRANT
GRANT
GRANT
GRANT
GRANT
```

Capture 19 : roles.sql sur queenbee_test

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/4_test_concurrence/jeton.sql
psql:jour2/4_test_concurrence/jeton.sql:1: NOTICE:  la table « session_app » n'existe pas, poursuite du traitement
psql:jour2/4_test_concurrence/jeton.sql:1: NOTICE:  la table « jeton » n'existe pas, poursuite du traitement
DROP TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 30

junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/5_revue_securite/corrections.sql
psql:jour2/5_revue_securite/corrections.sql:2: NOTICE:  la vue « v_carte_ruches » n'existe pas, poursuite du traitement
DROP VIEW
CREATE VIEW
GRANT
ALTER VIEW
ALTER VIEW
ALTER VIEW
ALTER VIEW
REVOKE
GRANT
GRANT
ALTER TABLE
psql:jour2/5_revue_securite/corrections.sql:28: NOTICE:  la politique « recolte_apicultrice_isolement » de la relation « recolte » n'existe pas, poursuite du
 traitement
DROP POLICY
CREATE POLICY
GRANT
```

Capture 20 : jeton.sql, puis corrections.sql sur queenbee_test

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -v ON_ERROR_STOP=1 -f jour2/5_revue_securite/corrections.sql
DROP VIEW
CREATE VIEW
GRANT
ALTER VIEW
ALTER VIEW
ALTER VIEW
ALTER VIEW
REVOKE
GRANT
GRANT
ALTER TABLE
DROP POLICY
CREATE POLICY
GRANT
UPDATE 1
UPDATE 1
INSERT 0 1
INSERT 0 1
```

Capture 21 : corrections.sql relancé : aucune notice ni erreur (rejouable)

Question 4 : refaire les tests de contournement sur la base reconstruite

> *Attendu : aucune ERROR, puis les tests de contournement donnent les mêmes résultats que sur votre base de travail.*

Réponse. on a lancé tests_contournement.sql sur queenbee_test, sans ON_ERROR_STOP car les refus sont voulus. Les 12 tests donnent le résultat attendu par le TP. Les messages sont ceux de PostgreSQL, en français.

N°	Qui	Tentative	Résultat obtenu
1	f.ba	lire apicultrice directement	droit refusé pour la table apicultrice
2	f.ba	lire v_mes_ruches	1 ligne : SN-KOLD-012 (ses ruches seulement)
3	f.ba	déclarer une récolte sur sa ruche	INSERT 0 1
4	f.ba	déclarer une récolte sur la ruche d'une autre	la nouvelle ligne viole la contrainte de vérification pour la vue « v_mes_recoltes »
5	f.ba	déclarer une récolte déjà validée	droit refusé pour la vue v_mes_recoltes
6	f.ba	modifier un volume directement	droit refusé pour la table recolte
7	f.ba	appeler valider_recolte	droit refusé pour la fonction valider_recolte
8	f.ba	écrire une fausse ligne dans le journal	droit refusé pour la table journal_audit
9	f.ba	désactiver le trigger d'audit	doit être le propriétaire de la table recolte

N°	Qui	Tentative	Résultat obtenu
10	coord.sn	valider la récolte du test 3, puis la revalider	première validation acceptée (1 ligne) ; seconde refusée : récolte introuvable, déjà traitée ou hors de votre site
11	coord.sn	valider une récolte d'un autre site (CM-BAF)	refusé : récolte introuvable, déjà traitée ou hors de votre site
12	admin.beeafrica	lire le journal, puis supprimer un utilisateur	lecture : 8 lignes ; suppression : droit refusé pour la table utilisateur

Le journal lu par l'administrateur contient f.ba | recolte | INSERT et coord.sn | recolte | UPDATE : ce sont les vrais acteurs, même quand la validation passe par une fonction exécutée avec les droits de postgres (SECURITY DEFINER avec qui()).

```
junel@Junel MINGW64 /d/Mon travail/TP/tp postgree/postgree_queenbee (junel)
$ psql -U postgres -d queenbee_test -f jour2/2_roles_privileges/tests_contournement.sql
INSERT 0 1
--- test 1 : f.ba lit apicultrice directement
SET
psql:jour2/2_roles_privileges/tests_contournement.sql:15: ERREUR:  droit refusé pour la table apicultrice
--- test 2 : f.ba lit v_mes_ruches
     nom
-------------
 SN-KOLD-012
(1 row)


--- test 3 : f.ba dÃ©clare une rÃ©colte sur sa ruche
INSERT 0 1
--- test 4 : f.ba dÃ©clare une rÃ©colte sur la ruche d une autre
psql:jour2/2_roles_privileges/tests_contournement.sql:32: ERREUR:  la nouvelle ligne viole la contrainte de vérificat
ion pour la vue « v_mes_recoltes »
--- test 5 : f.ba dÃ©clare une rÃ©colte dÃ©jÃ  validÃ©e
psql:jour2/2_roles_privileges/tests_contournement.sql:38: ERREUR:  droit refusé pour la vue v_mes_recoltes
--- test 6 : f.ba modifie un volume directement
psql:jour2/2_roles_privileges/tests_contournement.sql:43: ERREUR:  droit refusé pour la table recolte
--- test 7 : f.ba appelle valider_recolte
psql:jour2/2_roles_privileges/tests_contournement.sql:48: ERREUR:  droit refusé pour la fonction valider_recolte
--- test 8 : f.ba Ã©crit une fausse ligne dans le journal
psql:jour2/2_roles_privileges/tests_contournement.sql:54: ERREUR:  droit refusé pour la table journal_audit
--- test 9 : f.ba dÃ©sactive le trigger d audit
psql:jour2/2_roles_privileges/tests_contournement.sql:59: ERREUR:  doit être le propriétaire de la table recolte
RESET
--- test 10 : coord.sn valide la rÃ©colte du test 3, puis la revalide
SET
 valider_recolte
```

Capture 22 : tests_contournement.sql sur queenbee_test (tests 1 à 10)

```
SET
 valider_recolte
-----------------

(1 row)


psql:jour2/2_roles_privileges/tests_contournement.sql:70: ERREUR:  rÃ©colte introuvable, dÃ©jÃ  traitÃ©e ou hors de v
otre site
CONTEXT:  fonction PL/pgSQL valider_recolte(uuid), ligne 22 à RAISE
--- test 11 : coord.sn valide une rÃ©colte d un autre site (CM-BAF)
psql:jour2/2_roles_privileges/tests_contournement.sql:75: ERREUR:  rÃ©colte introuvable, dÃ©jÃ  traitÃ©e ou hors de v
otre site
CONTEXT:  fonction PL/pgSQL valider_recolte(uuid), ligne 22 à RAISE
RESET
--- test 12 : admin.beeafrica lit le journal, puis supprime un utilisateur
SET
  acteur  | table_cible | operation
----------+-------------+-----------
 postgres | utilisateur | INSERT
 postgres | utilisateur | UPDATE
 postgres | utilisateur | UPDATE
 postgres | utilisateur | INSERT
 postgres | utilisateur | INSERT
 postgres | recolte     | INSERT
 f.ba     | recolte     | INSERT
 coord.sn | recolte     | UPDATE
(8 rows)


psql:jour2/2_roles_privileges/tests_contournement.sql:83: ERREUR:  droit refusé pour la table utilisateur
RESET
```

Capture 23 : tests_contournement.sql sur queenbee_test (fin du test 10, tests 11 et 12)

Étape 11.6 · Archiver et rendre

Question 1 : commit et archive du dossier

> *git add -A, git commit -m "Jour 2 : dossier de conception", puis git archive --format=zip -o GROUPE_X_jour2.zip HEAD jour2.*

Réponse. le rangement est fait (21 fichiers renommés) et le schéma schema_j2.sql est exporté. git add -A prendra les modifications non indexées des quatre fichiers RM et le fichier non suivi schema_j2.sql (capture 12).

Synthèse : attendu et obtenu

Étape	Attendu	Obtenu
11.1 export du schéma	recolte_audit, security_barrier et GRANT sur v_mes_ruches dans schema_j2.sql	trois motifs trouvés (lignes 826, 474 à 569 et 1398)
11.2 tests_audit.sql	12.5 puis 14.0 ; trois refus ; limite du propriétaire	12.50 puis 14.00 ; trois refus, journal à 2 lignes ; deux ALTER TABLE
11.3 rangement	git mv sans erreur	21 renommages, aucun « not under version control »
11.4 README	jour2/README.md	Dans le code
11.5 reconstruction	aucune ERROR	aucune erreur, notices seulement ; corrections.sql rejouable
11.5 tests de contournement	mêmes résultats que la base de travail	12 tests conformes ; comparaison avec la base de travail à ajouter
11.6 archive	zip du dossier jour2	-